REPUBLIC OF THE PHILIPPINES

BATANGAS STATE UNIVERSITY

The National Engineering University

ARASOF-Nasugbu Campus

R. Martinez St., Brgy. Bucana, Nasugbu, Batangas, Philippines 4231

College of Informatics and Computing Sciences

Face Recognition Attendance System using Flask, OpenCV, Docker, Kubernetes, and Ngrok

Updated system documentation based on the current project structure

Ata, Lira Angela

Creencia, Eloisa Joyce

Buenaobra, Allysa

Boiser, Jhon Paulo

Nasugbu, Batangas

2026

Semestral Project in IT 323 and NTT 404 | Second Semester, AY 2025-2026

System Overview

The updated system is a web-based Face Recognition Attendance System that automates student attendance through a browser camera, a Flask backend, and OpenCV-based face processing. The current implementation does not use YOLOv8 or MySQL. It uses OpenCV Haar Cascade for face detection, OpenCV LBPH Face Recognizer for recognition, and SQLite for persistent attendance data.

The application is containerized with Docker, served by Gunicorn, deployed to Kubernetes or Minikube, and exposed publicly for demonstrations using ngrok. Runtime data such as the SQLite database, registered face samples, attendance snapshots, and trained recognition model are stored in the application instance directory and mounted into Kubernetes through a PersistentVolumeClaim.

Purpose of the System

- Automate attendance recording using live camera-based face recognition.
- Provide an admin-controlled web interface for registering students and reviewing records.
- Demonstrate containerized deployment using Docker.
- Demonstrate Kubernetes deployment using Deployment, Service, Secret, and PersistentVolumeClaim resources.
- Expose the local Kubernetes service through ngrok for HTTPS browser access during presentation.

Main Features

User and Admin Features

- Admin login and protected dashboard pages.
- Default admin creation on first run, configurable through environment variables or Kubernetes Secrets.
- Student registration with live camera capture and at least three face samples.
- Auto-generated Student ID values such as `STU-0001`, `STU-0002`, and following numbers.
- Course chooser limited to `BSIT-NT 3201` and `BSIT-NT 3202`.
- Live attendance recognition through the browser camera.
- Attendance dashboard, recent activity summary, searchable records, and CSV export.

Computer Vision Features

- OpenCV Haar Cascade face detection.
- OpenCV LBPH face recognition model.
- Configurable face match threshold through `FACE_MATCH_THRESHOLD`.
- Automatic model retraining after student registration.
- Saved normalized student face samples and attendance snapshots.

Current Project Structure

Path	Purpose
`app.py`	Main Flask application, routes, REST APIs, admin authentication, registration, attendance recognition, CSV export, and CLI commands.
`database.py`	SQLite schema and database connection helpers for admins, students, face samples, and attendance logs.
`face_engine.py`	OpenCV face detection, registration preprocessing, LBPH recognition, snapshot saving, and model training.
`templates/`	Flask HTML templates for login, setup, dashboard, register user, live attendance, and records pages.
`static/js/script.js`	Browser camera handling, registration captures, attendance scan requests, dashboard rendering, and records table updates.
`static/css/input.css` and `static/css/styles.css`	Tailwind source CSS and compiled application styles.
`Dockerfile`	Python 3.12 slim image definition with Flask, Gunicorn, and OpenCV dependencies.
`k8s/attendance-system.yaml`	Production Kubernetes manifest using the GHCR image, Namespace, Deployment, Service, and PVC.
`k8s/attendance-system-minikube.yaml`	Local Minikube manifest using the local Docker image and a Minikube-friendly PVC.
`k8s/attendance-system-secret.example.yaml`	Safe template for required admin and secret configuration.
`requirements.txt`	Python dependencies: Flask, Gunicorn, and OpenCV contrib headless.

System Components

1. **Frontend:** HTML templates, compiled Tailwind CSS, and JavaScript camera workflows.
2. **Backend:** Flask routes and JSON APIs running under Gunicorn on port 5000.
3. **Database:** SQLite file stored at `/app/instance/attendance.sqlite3`.
4. **Computer Vision:** OpenCV Haar Cascade detection and LBPH recognition.
5. **Container:** Docker image built from `python:3.12-slim`.
6. **Orchestration:** Kubernetes Deployment, Service, Secret, and PersistentVolumeClaim.
7. **Public Access:** ngrok forwards localhost to an HTTPS URL for camera-friendly browser testing.

Database Design

Table	Stored Data
`admins`	Admin email, hashed password, and creation timestamp.
`students`	Generated student number, full name, course, active status, and timestamps.
`face_samples`	File paths for registered face images linked to a student.
`attendance_logs`	Date, time, status, confidence score, and optional attendance snapshot path.

Recognition Workflow

1. The admin logs in and opens the Register User page.
2. The browser camera captures at least three face samples for the student.
3. The system generates the next Student ID automatically and saves the selected course.
4. OpenCV prepares the detected face regions and saves normalized samples.
5. The LBPH model is retrained using active students' saved face samples.
6. During attendance, the browser sends camera frames to the recognition API.
7. If a face matches a registered student above the threshold, attendance is logged once per day.

Important API Endpoints

Endpoint	Purpose
`GET /healthz`	Kubernetes liveness check.
`GET /readyz`	Kubernetes readiness check and database availability check.
`GET /api/students`	Returns registered students and sample counts.
`GET /api/students/next-id`	Returns the next generated Student ID.
`POST /api/students`	Registers a student, saves face samples, and retrains the model.
`POST`	Recognizes a face from a camera frame and records attendance.

Endpoint	Purpose
`/api/attendance/recognize`	
`GET /api/records/export`	Exports attendance records to CSV.

Docker Configuration

The current Dockerfile uses Python 3.12 slim, installs OpenCV runtime libraries, installs dependencies from `requirements.txt`, exposes port 5000, and starts the Flask app using Gunicorn.

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --upgrade pip && pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--threads", "4", "--timeout", "120", "app:app"]
```

Local Docker Commands

```
docker build -t face-attendance-system:latest .

docker run --rm -p 5000:5000 -v "$PWD/instance:/app/instance" face-attendance-system:latest
```

Kubernetes Deployment

The Kubernetes deployment uses a Namespace, Secret, PersistentVolumeClaim, Deployment, and NodePort Service. The service exposes port 80 and forwards traffic to the Flask container on port 5000. Health checks use `/healthz` and `/readyz`.

Production Manifest

- Image: `ghcr.io/23-75125-pixel/iasccofficial:latest`
- Image pull policy: `Always`
- Replicas: `5`
- Persistent storage: `ReadWriteMany`, 5Gi
- Service: NodePort `30080`

- Secret values loaded from `face-attendance-secret`

Minikube Manifest

- Image: `face-attendance-system:latest`
- Image pull policy: `Never`
- Replicas: `5` in the current local manifest.
- Persistent storage: `ReadWriteOnce`, 2Gi
- Service: NodePort `30080`
- Used for local testing without pushing to a registry.

Minikube Run Commands

```
minikube start

docker build -t face-attendance-system:latest .

minikube image load face-attendance-system:latest

kubectl apply -f k8s/attendance-system-minikube.yaml

kubectl rollout status deployment/face-attendance-web -n face-attendance
```

Expose with Port Forward and Ngrok

```
kubectl port-forward -n face-attendance svc/face-attendance-service 5000:80

ngrok http 5000



# Preferred for 5 replicas:

ngrok http "$(minikube ip):30080"
```

System Architecture

The system follows a browser-to-Kubernetes web application architecture. The browser captures camera frames, sends them to the Flask API through the Kubernetes Service, and receives recognition results. The Flask container performs OpenCV detection and recognition, then stores attendance records and face data in the mounted instance directory.

```
Browser Camera | v ngrok HTTPS URL or localhost:5000 | v kubectl port-forward /
```

Figure 1. Updated system architecture based on the current project.

Security Features

- Admin authentication is required before using dashboard, registration, attendance, and records pages.
- Passwords are stored as hashed values using Werkzeug security helpers.
- Kubernetes Secrets store default admin credentials and the Flask secret key.
- ngrok provides an HTTPS URL during demonstrations, which is useful for browser camera permissions.
- Attendance logs include timestamps and confidence values for audit review.

Expected Output

- Admin can log in and access the dashboard.
- Admin can register students with automatically generated Student IDs.
- Admin can choose only `BSIT-NT 3201` or `BSIT-NT 3202` during registration.
- Browser camera can capture student face samples and scan attendance.
- The system logs recognized students as Present with date, time, and confidence.
- Records can be searched and exported to CSV.
- The Docker image can run locally and inside Minikube/Kubernetes.
- The app can be exposed with ngrok for a public HTTPS demonstration link.

Approval

Approved by:

Mr. CALVIN JOHN V. PLACIO

Course Facilitator

Date Signed: _____